

1. 성능 최적화의 개요

딥러닝에서 성능 최적화란 단순히 모델을 학습시키는 것을 넘어, **데이터·알고리즘·하이퍼파라미터·하드웨어**를 종합적으로 조정하여 모델의 일반화 성능과 학습 효율을 극대화하는 과정을 의미한다. 성능 향상은 단일 기법으로 달성되기 어렵고, 반복적인 실험과 비교를 통해 점진적으로 이루어진다.

2. 데이터 기반 성능 최적화

딥러닝 모델의 성능은 데이터의 양과 질에 크게 의존한다.

- **데이터 수 증가:** 일반적으로 데이터가 많을수록 모델의 표현력이 향상되고 과적합 위험이 감소한다.
- **데이터 생성(Data Augmentation):** 충분한 데이터를 확보하기 어려운 경우, 기존 데이터를 변형하여 학습 데이터를 확장할 수 있다.
- **데이터 범위 조정:** 활성화 함수에 따라 입력 데이터의 범위를 조정하는 것이 중요하다.
예를 들어 시그모이드 함수 사용 시 입력 범위를 0~1 로, 하이퍼볼릭 탄젠트 함수 사용 시 -1~1 로 맞추는 것이 학습 안정성에 도움이 된다.
- **정규화·표준화:** 입력 데이터의 스케일을 통일함으로써 학습 속도와 성능을 향상시킨다.

3. 알고리즘 기반 성능 최적화

문제 유형에 따라 적합한 알고리즘을 선택하는 것이 중요하다.

- 분류 문제에서는 SVM, KNN, 신경망 등을 비교하여 최적 모델을 선택한다.
- 시계열 데이터의 경우 RNN, LSTM, GRU 등 다양한 구조를 실험해 성능을 평가한다.
- 하나의 알고리즘에 집착하기보다 **여러 후보 모델을 비교 실험**하는 것이 핵심 전략이다.

4. 하이퍼파라미터 튜닝을 통한 성능 최적화

하이퍼파라미터 튜닝은 성능 최적화에서 가장 많은 시간이 소요되는 단계이다.

- **모델 진단:**
 - 훈련 성능이 검증 성능보다 현저히 높으면 과적합 가능성이 크다.
 - 훈련·검증 성능 모두 낮으면 과소적합을 의심할 수 있다.
- **가중치 초기화:** 작은 난수 또는 사전 학습된 가중치를 사용하여 학습 안정성을 확보한다.
- **학습률(Learning Rate):** 네트워크가 깊을수록 상대적으로 큰 학습률이 필요하며, 너무 크거나 작으면 학습이 불안정해진다.
- **활성화 함수:** 활성화 함수 변경 시 손실 함수와의 조합을 함께 고려해야 한다.
- **배치 크기와 에포크:** 작은 배치와 큰 에포크가 최근 경향이지만, 데이터 크기에 따라 실험적 조정이 필요하다.
- **옵티마이저와 손실 함수:** SGD, Adam, RMSProp 등 다양한 옵티마이저를 비교하여 최적 조합을 선택한다.
- **네트워크 구조:**
 - 뉴런 수를 늘려 네트워크를 넓게 만들거나
 - 층 수를 늘려 깊게 구성하는 등 구조적 변화를 통해 성능을 개선한다.

5. 앙상블을 이용한 성능 최적화

앙상블 기법은 두 개 이상의 모델을 결합하여 사용하는 방법으로, 단일 모델보다 일반화 성능을 향상시킬 수 있다. 다만 하이퍼파라미터 조합 수가 기하급수적으로 증가하므로 많은 학습 시간이 요구된다.

6. 하드웨어 기반 성능 최적화

딥러닝에서는 하드웨어 선택이 학습 시간에 결정적인 영향을 미친다.

- **CPU vs GPU**
 - CPU 는 직렬 처리에 최적화되어 있으며, 개별 코어 성능이 높다.
 - GPU 는 다수의 ALU 를 활용한 병렬 처리에 특화되어 있어 딥러닝 학습에 매우 효율적이다.
- 역전파와 같은 대규모 행렬 연산은 GPU 에서 병렬 처리 시 학습 시간이 크게 단축된다.
- 대규모 데이터셋을 다루는 딥러닝에서는 GPU 사용이 선택이 아닌 필수에 가깝다.

7. CUDA 와 cuDNN 을 활용한 GPU 가속

- **CUDA** 는 NVIDIA 에서 개발한 병렬 연산 플랫폼으로, GPU 를 활용한 고속 연산을 가능하게 한다.
- **cuDNN** 은 딥러닝 연산에 특화된 CUDA 기반 라이브러리로, PyTorch·TensorFlow 등 주요 프레임워크에서 사용된다.
- PyTorch GPU 버전 설치를 통해 모델 학습을 GPU 에서 수행할 수 있으며, 이는 학습 속도와 실험 효율을 크게 향상시킨다.

8. 배치 정규화(Batch Normalization)

배치 정규화는 학습 과정에서 각 층의 입력 분포를 일정하게 유지하여 학습을 안정화하는 기법이다.

- 내부 공변량 변화(Internal Covariate Shift)를 완화한다.
- 기울기 소멸 및 기울기 폭발 문제를 완화한다.
- 일반적으로 **완전 연결층 또는 합성곱층 뒤, 활성화 함수 앞에** 위치한다.
- 학습 속도를 빠르게 하고 성능을 향상시키지만, 배치 크기가 너무 작을 경우 불안정해질 수 있다.

9. 드롭아웃(Dropout)

드롭아웃은 학습 시 일부 뉴런을 무작위로 비활성화하여 과적합을 방지하는 기법이다.

- 은닉층의 노드를 확률적으로 제거함으로써 특정 뉴런에 대한 의존을 줄인다.
- 학습 시간은 증가하지만 일반화 성능 향상에 효과적이다.
- 테스트 단계에서는 모든 노드를 사용하되 드롭아웃 비율을 고려해 출력을 조정한다.